```
  String author,

  @NotNull(message = "The book price must be defined.")
  @Positive(message = "The book price must be greater than zero.")
  Double price,

  @Version                                          ❷
  int version

){
  public static Book of(
    String isbn, String title, String author, Double price
  ) {
    return new Book(
      null, isbn, title, author, price, 0           ❸
    );
  }
}
```

❶ Identifies the field as the primary key for the entity

❷ The entity version number, which is used for optimistic locking

❸ An entity is considered new when the ID is null and the version is 0.

NOTE Spring Data JPA works with mutating objects, so you can't use Java records. JPA entity classes must be marked with the `@Entity` annotation and expose a no-args constructor. JPA identifiers are annotated with `@Id` and `@Version` from the `javax.persistence` package instead of `org.springframework.data.annotation`.

After adding the new fields, we need to update a few classes using the `Book` constructor, which now requires passing values for `id` and `version`.

The `BookService` class contains the logic for updating books. Open it and change the `editBookDetails()` method to ensure the book identifiers and versions are correctly passed along when calling the data layer.

Listing 5.5 Including existing identifier and version on book updates

```
package com.polarbookshop.catalogservice.domain;

@Service
```